Deduplicação de leituras de sensores: recomendação final por cliente

Relatório final · 10 de novembro de 2026 · Código e dados: github.com/matheus-pelissari/pjbl1-complexidade

Das dez estratégias do catálogo, duas foram implementadas e medidas. A **força bruta** é a única que cabe nos 256 B do sensor da Embarca e termina em tempo útil. O **Quicksort com varredura de vizinhos** processa 10^7 leituras em 0,84 s, abaixo do limite de 2 s da Datacenter. As outras oito foram colocadas no mesmo gráfico por extrapolação, e as quatro estratégias absurdas são rejeitadas com conta.

EMBARCA LTDA. · LIMITE: 256 B DE MEMÓRIA EXTRA

Estratégia 1 · Força bruta

48 B

de memória extra, igual para n = 10^3, 10^5 e 10^7

- 1 frame de pilha e 0 B de heap em todas as 15 rodadas.
- Exata e preserva a ordem de chegada das leituras.
- 58,0 s com n = 10^7. A Embarca não tem limite de tempo.
- Toda outra estratégia que cabe em 256 B de RAM é absurda (Bogosort, unária em disco).

DATACENTER S.A. · LIMITE: N = 10^7 EM MENOS DE 2 S

Estratégia 2 · Quicksort + varredura

0,84 s

mediana de 5 rodadas com n = 10^7 (de 0,72 a 0,91 s)

- Folga de 2,4× sobre o limite; a pior rodada ficou em 0,91 s.
- Exata. A saída sai ordenada, não na ordem de chegada.
- 2,5 KB de pilha de recursão e 0 B de heap.
- Se a ordem de chegada importar ou o prazo apertar, o bitmap (estratégia 5) é a evolução: ≈ 19 ms estimados.

1 O problema e as restrições

Eliminar leituras duplicadas de um vetor de leituras de sensor, entregando uma solução para cada cliente. As restrições abaixo vêm do enunciado e valem para a análise e para a implementação.

Elemento	Definição
Domínio dos dados	Inteiros de 16 bits, faixa fixa e conhecida: 0 a 65.535
Tamanhos de entrada	n = 10^3, 10^5 e 10^7
Embarca Ltda.	No máximo 256 bytes de memória extra, independentemente de n, além do próprio vetor de entrada
Datacenter S.A.	Sem limite de memória; n = 10^7 em menos de 2 segundos na máquina de referência
Correção	Exata; se a estratégia for probabilística, falso positivo declarado ≤ 1%

2 Por que estas duas estratégias

Embarca. O limite de 256 B elimina toda estratégia que guarda uma estrutura auxiliar. O bitmap, a menor delas, precisa de 8.192 B (32× o limite); Bloom, hash e árvore vão de dezenas de KB a 2,6 MB. O Quicksort passa do limite só com a pilha de recursão: 1,2 KB já com n = 10^3. Sobram as que usam apenas contadores: a força bruta, o Bogosort (≈ 10^{2554} anos com n = 10^3) e a unária em disco (41 GB de disco com n = 10^7). Só a força bruta termina sem depender de outro recurso.

Datacenter. Cinco estratégias cabem nos 2 s com n = 10^7: Quicksort (medido), bitmap, hash, Bloom e árvore (estimados). O Quicksort com varredura foi o implementado: é exato (o Bloom erra até 1%), não aloca nada no heap (hash e árvore alocam de 0,5 a 2,6 MB) e não depende de o domínio ser pequeno e conhecido, que é o que

torna o bitmap viável. Ele tem dois custos: a saída não preserva a ordem de chegada, e o bitmap seria cerca de 40× mais rápido pela estimativa.

O par. As duas estratégias ficam em extremos opostos: uma iterativa, com memória constante e tempo que cresce com n·k; outra recursiva, rápida, em que a pilha cresce com n. Nenhuma das duas atende ao outro cliente: a força bruta leva 58 s onde o limite é 2 s, e o Quicksort usa 2,5 KB onde o limite é 256 B.

3 O que foi medido

Os dois programas estão em Java (JDK 17). As entradas são uniformes em 0..65.535, geradas com semente fixa e iguais para as duas estratégias. Cada tamanho foi executado 5 vezes; a tabela traz a mediana e a faixa.

Estratégia	n	Tempo (mediana)	Faixa das 5 rodadas	Operações	Pilha	Heap	Únicos
🔵 **1 · Força bruta**	10^3	89,0 µs	88 a 97 µs	$4{,}96\times10^5$	48 B (1 frame)	0 B	987 a 995
	10^5	406,2 ms	396,6 a 445,1 ms	$2{,}26\times10^9$	48 B (1 frame)	0 B	51.140 a 51.412
	10^7	**58,0 s**	57,6 a 59,7 s	$3{,}27\times10^{11}$	**48 B** (1 frame)	0 B	65.536
🟠 **2 · Quicksort + varredura**	10^3	85,0 µs	68 a 108 µs	$2{,}04\times10^4$	1.152 B (20 a 27 frames)	0 B	987 a 995
	10^5	9,3 ms	9,1 a 10,1 ms	$3{,}22\times10^6$	2.064 B (38 a 47 frames)	0 B	51.140 a 51.412
	10^7	**844,0 ms**	716,9 a 914,6 ms	$4{,}36\times10^8$	**2.496 B** (49 a 53 frames)	0 B	65.536

Pilha: mediana das 5 rodadas; o máximo do Quicksort foi 2.544 B (53 frames). Operações: comparações + escritas, mediana.

Como cada grandeza foi obtida

Grandeza	Instrumentação
Tempo	`System.nanoTime` na versão sem contadores, após 30 execuções de aquecimento do JIT.
Heap	Bytes alocados pela thread durante a medição (`ThreadMXBean`).
Pilha	O número de frames é medido (profundidade máxima da recursão + 1). Os bytes são frames × 48 B, uma estimativa do tamanho de um frame; não é uma leitura direta da pilha.
Operações	Segunda execução, instrumentada, sobre a mesma entrada. Na força bruta com n = 10^7, contagem analítica exata, validada contra a instrumentada em 10^3 e 10^5.

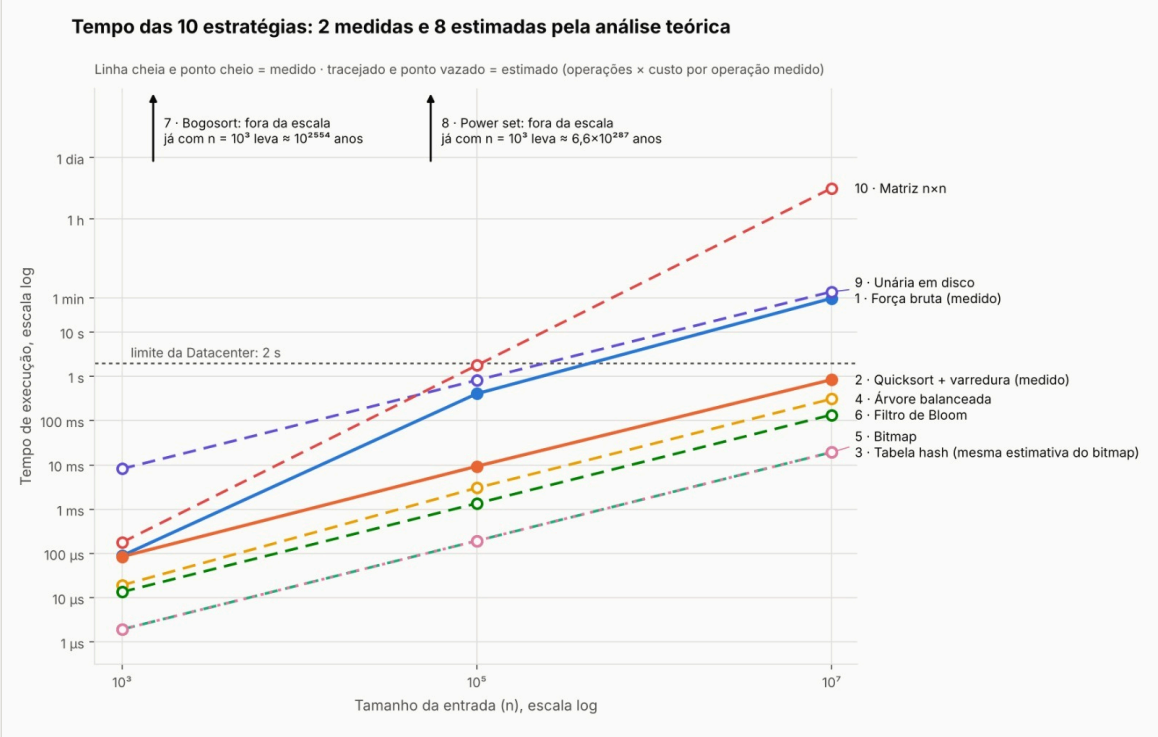
Casos de teste obrigatórios

Exigência	Resultado	Evidência
Corretude (n ≤ 10)	Passou	8 casos por estratégia, conferíveis à mão: vazio, 1 elemento, todos iguais, com e sem duplicatas, extremos da faixa e ordem inversa. Nas 30 rodadas de medição, o número de únicos confere com uma contagem independente.
Limite de memória (Embarca)	Passou	Força bruta: 48 B de pilha + 0 B de heap em todos os n, contra 256 B.
Orçamento de tempo (Datacenter)	Passou*	Quicksort + varredura: 844 ms de mediana com n = 10^7; as cinco rodadas ficaram abaixo de 2 s.
Validação das absurdas	Feita	Seção 6 e Anexo A: uma conta numérica por estratégia.
Coerência gráfica	Feita	Seções 4 e 5: as oito curvas e as divergências em relação à análise teórica.

> * **Máquina de referência.** Os tempos deste relatório foram medidos fora da máquina de referência da disciplina. A folga de 2,4× do Quicksort deve absorver a diferença, mas a medição precisa ser repetida lá, e os custos por operação usados nas extrapolações (c_A e c_B) recalculados. As contagens de operações, frames e bytes não dependem da máquina.

4 As dez estratégias no mesmo gráfico

As oito estratégias não implementadas foram posicionadas pela regra: **tempo estimado = operações previstas pela análise teórica × custo por operação medido**. Há duas réguas, tiradas das medições com n = 10^7: c_A = 1,94 ns (Quicksort: 844 ms ÷ 4,36×10^8 operações), para acesso espalhado, recursão e trocas; e c_B = 0,18 ns (força bruta: 58,0 s ÷ 3,27×10^{11} operações), para varredura sequencial. A unária usa a vazão de gravação de um SSD (500 MB/s).



Tempo de execução em função de n, em escala log-log. Linha cheia: medido. Tracejado: estimado. Bogosort e power set ficam fora da escala já com n = 10^3.

#	Estratégia	Tempo	Tempo com n = 10^7	Memória extra com n = 10^7	Embarca (≤ 256 B)	Datacenter (< 2 s)
1	**Força bruta**	O(n·k), k ≤ 65.536	58,0 s (medido)	48 B	Atende	Não: 29× o limite
2	**Quicksort + varredura**	Θ(n log n) médio	844 ms (medido)	2,5 KB de pilha	Não: ≈ 10×	Atende
3	Tabela hash	Θ(n) médio	≈ 19 ms	524 KB	Não: ≈ 2.000×	Atende
4	Árvore balanceada	Θ(n log k)	≈ 310 ms	2,6 MB	Não: ≈ 10.000×	Limítrofe (ver seção 5)
5	Bitmap	Θ(n)	≈ 19 ms	8,2 KB	Não: 32×	Atende
6	Filtro de Bloom	Θ(n·h), h = 7	≈ 136 ms	78,6 KB	Não: ≈ 300×	Atende, com até 1% de erro
7	Bogosort	Θ(n·n!) esperado	≈ $10^{48.067.074}$ anos	48 B	Não: cabe na memória, mas não termina	Não
8	Power set	Θ(n·2^n)	≈ $10^{3.010.291}$ anos	560 MB	Não	Não: estoura a pilha antes
9	Unária em disco	Θ(Σv_i)	≥ 82 s (só gravar)	48 B + 41 GB de disco	Não: cabe na RAM, exige disco	Não: 41× o limite
10	Matriz n×n	Θ(n^2)	≈ 4,9 h	12,5 TB	Não	Não: ≈ 8.900× o limite

5 Teoria e prática: onde divergem e por quê

A análise teórica acertou as **contagens** e errou alguns **custos unitários**. Nenhuma divergência muda a recomendação por cliente.

Estratégias medidas

Item ($n = 10^7$)	Previsto na análise teórica	Medido	Leitura
Força bruta, operações	$n \cdot k/2 \approx 3{,}3 \times 10^{11}$	$3{,}27 \times 10^{11}$	Confere, com diferença menor que 1%.
Força bruta, tempo	≈ 5,5 min (1 ns por comparação)	58,0 s	5,6× mais rápido: cada operação custou 0,18 ns.
Força bruta, memória	≈ 13 B (variáveis locais)	48 B (1 frame)	O frame inclui mais que as variáveis. Os dois valores cabem em 256 B.
Quicksort, operações	≈ $3{,}2 \times 10^8$ comparações ($1{,}39 \cdot n \cdot \log_2 n$)	$4{,}36 \times 10^8$ comparações + escritas	Compatível: entre 1,9 e $2{,}0 \cdot n \cdot \log_2 n$ nos três tamanhos.
Quicksort, tempo	0,3 a 1 s	0,84 s	Dentro da faixa prevista.
Quicksort, pilha	≈ 1 KB (24 níveis × 40 B)	2,5 KB (até 53 frames × 48 B)	A recursão foi cerca de 2× mais funda que $\log_2 n$.

A força bruta foi 5,6× mais rápida que o previsto. A contagem de comparações estava certa; a análise superestimou o custo de cada uma. O laço interno percorre em sequência uma lista de no máximo 65.536 inteiros (256 KB), que cabe na cache do processador, e o acesso é previsível. O custo medido ficou praticamente constante nos três tamanhos: 0,180, 0,180 e 0,178 ns por operação.

A pilha do Quicksort foi 2,5× a prevista. A análise assumiu divisão sempre ao meio ($\log_2 n \approx 24$ níveis). Com pivô do meio sobre dados aleatórios as partições saem desiguais, e a profundidade medida ficou entre 1,9 e 2,8 vezes $\log_2 n$. O veredito não muda: a Embarca já era inviável com 1 KB, e com 2,5 KB a distância passa de 4× para 10×.

O custo por operação varia com o tamanho. O Quicksort custou 4,2 ns por operação com $n = 10^3$, 2,9 ns com 10^5 e 1,9 ns com 10^7. As curvas estimadas usam o valor de $n = 10^7$, então devem ser lidas em ordem de grandeza, e os pontos de $n = 10^3$ podem estar subestimados em até 2×.

Estratégias extrapoladas

Estratégia ($n = 10^7$)	Análise teórica	Curva do gráfico	Motivo da diferença
3 · Tabela hash	0,05 a 0,5 s	≈ 19 ms	A curva conta 1 operação por leitura, igual ao bitmap. Não inclui cálculo do hash, sondagem nem boxing. É um piso; o veredito é o mesmo.
4 · Árvore balanceada	0,5 a 3 s, limítrofe	≈ 310 ms	c_A vem do Quicksort, que percorre a memória quase em sequência. A árvore salta entre nós espalhados em 2,6 MB, e cada comparação tende a custar mais. A curva é otimista; mantemos o veredito de limítrofe.
10 · Matriz n×n	≈ 28 h (1 ns por operação)	≈ 4,9 h (c_B = 0,18 ns)	Mesma contagem (10^{14}). Muda só o custo por operação, que passou a ser o medido. Inviável nos dois casos.
5 · Bitmap	10 a 30 ms	≈ 19 ms	Confere.
6 · Filtro de Bloom	0,1 a 0,3 s	≈ 136 ms	Confere.
9 · Unária em disco	82 s a 11 min	≈ 82 s	Confere. A curva usa o caso mais favorável: 1 bit por símbolo, só a gravação.
7 · Bogosort e 8 · Power set	≈ 10^{2554} anos e 10^{301} subconjuntos com $n = 10^3$	Fora da escala	Confere.

Efeitos de ambiente

Com n = 10^7, as rodadas do Quicksort variaram de 717 a 915 ms (de −15% a +8% em torno da mediana) e as da força bruta, de 57,6 a 59,7 s (no máximo 3% de diferença). Por isso o relatório usa a mediana de 5 rodadas, com aquecimento do JIT antes de medir. Numa reexecução em outro ambiente (contêiner Linux, OpenJDK 21), as contagens de operações, únicos e frames foram idênticas nas 30 rodadas, e os tempos com n = 10^5 e 10^7 ficaram de 25% a 35% maiores: 1,05 s para o Quicksort e 75 s para a força bruta com n = 10^7. Os vereditos se mantêm.

6 Validação das estratégias absurdas

As quatro resolvem o problema corretamente, e cada uma estoura um recurso diferente por várias ordens de grandeza. O documento completo, com as deduções, está no Anexo A.

#	Estratégia	Recurso que estoura	Conta-chave	Quanto passa do limite	Deixaria de ser absurda com
7	**Bogosort**	Tempo, nos dois clientes	n = 10^3: 1000! ≈ $4{,}0\times10^{2567}$ embaralhamentos em média	≈ 10^{2554} anos, 10^{2544} vezes a idade do universo	n ≤ 11 na Datacenter (≈ 0,85 s)
8	**Power set**	Pilha na Embarca; tempo nos dois	n = 10^3: 2^{1000} ≈ $1{,}07\times10^{301}$ subconjuntos e ≈ 56 KB de memória extra	≈ 219× os 256 B; ≈ $6{,}6\times10^{287}$ anos	n ≤ 3 na Embarca; n ≤ 25 na Datacenter (≈ 1,6 s)
9	**Unária em disco**	Disco na Embarca; tempo na Datacenter	n = 10^7: $3{,}3\times10^{11}$ símbolos = 41 GB só de gravação	82 s contra 2 s (≈ 41×)	n ≤ ≈ 244 mil na Datacenter; valores muito pequenos
10	**Matriz n×n**	Memória na Embarca; tempo e memória na Datacenter	n = 10^3: 10^6 bits = 125 KB. n = 10^7: 10^{14} bits = 12,5 TB	≈ 488× os 256 B; 4,9 h contra 2 s	n ≤ 40 na Embarca; n ≤ ≈ 106 mil na Datacenter

Quantas permutações o Bogosort precisa com n = 1000. Com n valores distintos há n! arrumações e só uma está ordenada, então o número médio de tentativas é n!: 1000! ≈ $4{,}02\times10^{2567}$. Com as repetições das entradas reais (k ≈ 992) o valor cai cerca de duas casas decimais, para ≈ 10^{2565}.

O que cada uma ensina. O Bogosort mostra que espaço O(1) não basta: cabe em 48 B e não termina. O power set mostra a explosão de resolver um problema mais geral que o pedido, e que a pilha também é memória. A unária mostra que o Big-O de espaço precisa dizer qual recurso mede: ela economiza RAM empurrando o custo para o disco. A matriz mostra que gastar memória que ninguém pediu é tão errado quanto economizar demais.

7 Dados completos das dez estratégias

Uma linha por estratégia e tamanho. Fundo azul: medido. Demais: estimado por operações × régua. A coluna do Datacenter só se aplica a n = 10^7.

#	Estratégia	n	Tipo	Operações	Régua	Tempo	$\log_{10}$ do tempo (s)	Memória extra	Memória (bytes)	Disco	Embarca ≤ 256 B	Datacenter < 2 s	Observação
1	**Força bruta**	10^3	medido	$4{,}958\times10^5$	—	89,0 µs	−4,051	48 B	48		sim		
		10^5	medido	$2{,}258\times10^9$	—	406,2 ms	−0,391	48 B	48		sim		
		10^7	medido	$3{,}266\times10^{11}$	—	58,0 s	1,763	48 B	48		sim	não	
2	**Quicksort + varredura**	10^3	medido	$2{,}038\times10^4$	—	85,0 µs	−4,071	1,2 KB	1.152		não		
		10^5	medido	$3{,}223\times10^6$	—	9,3 ms	−2,030	2,1 KB	2.064		não		
		10^7	medido	$4{,}361\times10^8$	—	844,0 ms	−0,074	2,5 KB	2.496		não	sim	
3	**Tabela hash**	10^3	estimado	$1{,}000\times10^3$	c_A	1,9 µs	−5,713	8,2 KB	8.240		não		
		10^5	estimado	$1{,}000\times10^5$	c_A	193,5 µs	−3,713	524,3 KB	524.336		não		
		10^7	estimado	$1{,}000\times10^7$	c_A	19,4 ms	−1,713	524,3 KB	524.336		não	sim	
4	**Árvore balanceada**	10^3	estimado	$9{,}955\times10^3$	c_A	19,3 µs	−4,715	39,7 KB	39.745		não		
		10^5	estimado	$1{,}565\times10^6$	c_A	3,0 ms	−2,519	2,1 MB	2.051.515		não		
		10^7	estimado	$1{,}600\times10^8$	c_A	309,6 ms	−0,509	2,6 MB	2.621.488		não	sim	
5	**Bitmap**	10^3	estimado	$1{,}000\times10^3$	c_A	1,9 µs	−5,713	8,2 KB	8.240		não		
		10^5	estimado	$1{,}000\times10^5$	c_A	193,5 µs	−3,713	8,2 KB	8.240		não		
		10^7	estimado	$1{,}000\times10^7$	c_A	19,4 ms	−1,713	8,2 KB	8.240		não	sim	
6	**Filtro de Bloom**	10^3	estimado	$7{,}000\times10^3$	c_A	13,5 µs	−4,868	1,2 KB	1.238		não		1% de falso positivo
		10^5	estimado	$7{,}000\times10^5$	c_A	1,4 ms	−2,868	61,5 KB	61.496		não		1% de falso positivo
		10^7	estimado	$7{,}000\times10^7$	c_A	135,5 ms	−0,868	78,6 KB	78.569		não	sim	1% de falso positivo
7	**Bogosort**	10^3	estimado	10^{2571}	c_A	$\approx 10^{2554}$ anos	2.561,891	48 B	48		sim		tempo médio esperado
		10^5	estimado	$10^{447.953}$	c_A	$\approx 10^{447.937}$ anos	447.944,542	48 B	48		sim		tempo médio esperado
		10^7	estimado	$10^{48.067.090}$	c_A	$\approx 10^{48.067.074}$ anos	48.067.081,605	48 B	48		sim	não	tempo médio esperado
8	**Power set**	10^3	estimado	10^{304}	c_A	$\approx 6{,}6\times10^{287}$ anos	295,317	56,0 KB	56.048		não		
		10^5	estimado	$10^{30.108}$	c_A	$\approx 10^{30.092}$ anos	30.099,286	5,6 MB	5.600.048		não		estoura a pilha (StackOverflowError)
		10^7	estimado	$10^{3.010.307}$	c_A	$\approx 10^{3.010.291}$ anos	3.010.298,243	560,0 MB	560.000.048		não	não	estoura a pilha (StackOverflowError)
9	**Unária em disco**	10^3	estimado	$3{,}277\times10^7$	disco 500 MB/s	8,2 ms	−2,087	48 B	48	4,1 MB	não		só a gravação; 1 bit por símbolo
		10^5	estimado	$3{,}277\times10^9$	disco 500 MB/s	819,2 ms	−0,087	48 B	48	409,6 MB	não		só a gravação; 1 bit por símbolo
		10^7	estimado	$3{,}277\times10^{11}$	disco 500 MB/s	81,9 s	1,913	48 B	48	41,0 GB	não	não	só a gravação; 1 bit por símbolo
10	**Matriz n×n**	10^3	estimado	$1{,}000\times10^6$	c_B	177,6 µs	−3,751	125,0 KB	125.048		não		
		10^5	estimado	$1{,}000\times10^{10}$	c_B	1,8 s	0,249	1,3 GB	1.250.000.048		não		
		10^7	estimado	$1{,}000\times10^{14}$	c_B	4,9 h	4,249	12,5 TB	12.500.000.000.048		não	não	

Estratégia 9: a RAM extra (48 B) cabe nos 256 B; o "não" indica que ela exige um disco que o sensor não tem. Estratégia 7: o "sim" vale só para a memória; o tempo é inviável. Bogosort com n = 10^5 e 10^7: assume o mesmo número de cópias de cada valor, o que dá um limite superior do tempo esperado.

Fase 7: validação das estratégias absurdas

Nov 9, 2026 · @Matheus

As quatro estratégias absurdas resolvem o problema corretamente, mas cada uma estoura um recurso diferente por várias ordens de grandeza. Abaixo, cada rejeição vem com a conta, o cliente e o recurso violado.

Resumo

Nenhuma das quatro atende a qualquer cliente nos tamanhos testados. Cada uma estoura um recurso diferente.

#	Estratégia	Recurso que estoura	Conta-chave	Quanto passa do limite
7	Bogosort	Tempo (os dois clientes)	$n = 10^3$: $\approx 4{,}0 \times 10^{2567}$ embaralhamentos	$\approx 10^{2554}$ anos; 10^{2544} vezes a idade do universo
8	Power set	Pilha (Embarca) e tempo (os dois)	$n = 10^3$: $2^{1000} \approx 10^{301}$ subconjuntos e ≈ 56 KB de pilha	Pilha ≈ 219× os 256 B; tempo ≈ $6{,}6 \times 10^{287}$ anos
9	Unária em disco	Disco (Embarca) e tempo (Datacenter)	$n = 10^7$: $3{,}3 \times 10^{11}$ símbolos = 41 GB só de gravação	82 s contra 2 s (≈ 41×)
10	Matriz n×n	Memória (Embarca) e tempo e memória (Datacenter)	$n = 10^3$: 10^6 bits = 125 KB; $n = 10^7$: 10^{14} bits = 12,5 TB	≈ 488× os 256 B; 4,9 h contra 2 s

Os tempos usam o custo por operação medido nas duas estratégias implementadas (ver Premissas, no fim).

7 · Bogosort + varredura

Com n = 1000, o Bogosort precisa em média de $1000! \approx 4{,}0 \times 10^{2567}$ embaralhamentos, o que levaria $\approx 10^{2554}$ anos. Cabe na memória da Embarca, mas nunca termina.

Resposta à pergunta da seção 10. Com n valores distintos existem n! arrumações e só uma está ordenada. Cada embaralhamento acerta com probabilidade 1/n!, então o número médio de tentativas é n!:

$1000! \approx 4{,}02 \times 10^{2567}$ permutações em média

Nas entradas reais do trabalho há algumas repetições (k ≈ 992 com n = 1000). Com m_i cópias do valor i, o número esperado vira $n!/\prod m_i!$. Isso só tira cerca de 2 das 2.568 casas decimais: continua ≈ 10^{2565}.

Item	$n = 10^3$	$n = 10^7$
Tentativas esperadas	≈ $4{,}0 \times 10^{2567}$	≈ 10^48.067.083 (multinomial, ~150 cópias de cada valor)
Custo de cada tentativa	~n operações (embaralhar + conferir)	~n operações
Tempo esperado (× 1,94 ns)	≈ $7{,}8 \times 10^{2561}$ s ≈ $2{,}5 \times 10^{2554}$ anos	≈ 10^48.067.074 anos
Memória extra	48 B (1 frame, sem heap)	48 B

- **Embarca: inaceitável.** A memória cabe (48 B de 256 B), mas o tempo esperado é 10^{2544} vezes a idade do universo ($1{,}38 \times 10^{10}$ anos). Não há limite de tempo explícito, mas um algoritmo que não termina não entrega resultado.
- **Datacenter: inaceitável.** Com $n = 10^7$ o tempo tem mais de 48 milhões de dígitos.
- **Pior caso: ilimitado.** Nada garante que a arrumação certa saia algum dia.

Quando deixaria de ser absurdo: com $n \leqslant 11$, o tempo esperado ($11 \times 11! \times 1{,}94$ ns ≈ 0,85 s) cabe nos 2 s da Datacenter. Com n = 12 já passa para 11 s, e com n = 25 iguala a idade do universo. Também seria viável se quase todos os valores fossem iguais, porque $\prod m_i!$ fica enorme e poucas tentativas bastam.

O que ensina: espaço O(1) não basta. Uma estratégia pode caber em 48 B e ainda assim ser inaceitável no tempo, ao contrário do Quicksort, que resolve o mesmo problema em n log n.

8 · Enumeração recursiva de subconjuntos (power set)

Com n = 1000 já são $2^{1000} \approx 1{,}07 \times 10^{301}$ subconjuntos e ≈ 56 KB de pilha: estoura a memória da Embarca (≈ 219×) e o tempo dos dois clientes.

A recursão decide "entra ou não entra" para cada leitura, então gera 2^n subconjuntos e confere cada um em ~n operações. A pilha desce n níveis antes de completar o primeiro subconjunto.

$T(n) = 2 \cdot T(n-1) + O(1) \Rightarrow \Theta(2^n)$ chamadas e $\Theta(n \cdot 2^n)$ operações

Item	$n = 10^3$	$n = 10^7$
Subconjuntos	$2^{1000} \approx 1{,}07 \times 10^{301}$	$2^{(10^7)} \approx 10^{3.010.300}$
Tempo ($n \times 2^n \times 1{,}94$ ns)	$\approx 2{,}1 \times 10^{295}$ s $\approx 6{,}6 \times 10^{287}$ anos	$\approx 10^{3.010.291}$ anos
Pilha (48 B por nível)	1.001 frames ≈ 48 KB	10^7 frames ≈ 480 MB
Heap (subconjunto atual e melhor, int[])	≈ 8 KB	≈ 80 MB
Total extra	≈ 56 KB	≈ 560 MB

- **Embarca: inaceitável.** 56 KB contra 256 B já com $n = 10^3$ (≈ 219×). A memória extra é $48 \times (n + 1) + 8n$ bytes, e isso só cabe em 256 B com **$n \leqslant 3$**.
- **Datacenter: inaceitável.** O tempo é astronômico já com $n = 10^3$. Com $n = 10^5$ ou 10^7 o programa nem chega a calcular: a pilha padrão de uma thread Java (~1 MB) aguenta só uns 2×10^4 níveis de 48 B, e a execução termina em `StackOverflowError`.

Quando deixaria de ser absurdo: com $n \leqslant 25$, o tempo ($25 \times 2^{25} \times 1{,}94$ ns ≈ 1,6 s) cabe nos 2 s da Datacenter, e a pilha fica em ~1,4 KB. Com n = 26 já passa para 3,4 s, e com n = 82 iguala a idade do universo. Também faria sentido se o problema de fato pedisse subconjuntos (ex.: escolher o melhor grupo de sensores).

O que ensina: resolver um problema mais geral do que o necessário causa explosão combinatória. E a recursão profunda conecta tempo e memória: a pilha também é memória, e é ela que estoura primeiro.

9 · Codificação unária em disco

Com $n = 10^7$, só gravar as leituras em unário ocupa 41 GB de disco e leva ≈ 82 s, 41× o limite da Datacenter. Na Embarca, a RAM cabe, mas o custo foi empurrado para um disco que o sensor não tem.

Cada leitura x vira x símbolos no disco, então o volume gravado é a soma dos valores (Σv_i), não n. Com leituras uniformes em 0..65535, a média é 32.767,5 símbolos por leitura. A conta abaixo considera só a gravação, com 1 bit por símbolo: qualquer etapa de comparação só aumenta o custo.

Item	$n = 10^3$	$n = 10^7$
Símbolos gravados ($\approx 32.767{,}5 \times n$)	$\approx 3{,}3 \times 10^7$	$\approx 3{,}3 \times 10^{11}$

Item	$n = 10^3$	$n = 10^7$
Disco (1 bit por símbolo)	≈ 4,1 MB	≈ 41 GB
Disco (1 byte por símbolo, como texto)	≈ 33 MB	≈ 328 GB
Tempo só de gravação (500 MB/s)	≈ 8,2 ms	≈ 82 s (bits) a ≈ 655 s (bytes)
RAM extra (contadores)	48 B	48 B

- **Embarca: inaceitável.** Cumpre os 256 B de RAM (48 B), mas viola a premissa de que a memória do sensor é a RAM: ele não tem dezenas de GB de armazenamento, e tanta escrita desgastaria a memória flash. Com 1 MB de flash, por exemplo, caberiam só ≈ 244 leituras.
- **Datacenter: inaceitável.** Com $n = 10^7$, ≈ 82 s só de gravação, de 41× a 328× o limite de 2 s.
- **Melhor e pior caso:** dependem dos valores, não só de n (custo pseudo-polinomial). Todos 0: só n separadores. Todos 65.535: o dobro da média.

Quando deixaria de ser absurda: na Datacenter, com $n \leqslant \approx 244$ mil leituras a gravação cabe nos 2 s (com $n = 10^5$ seriam ≈ 0,82 s e 410 MB de disco). Também faria sentido com valores muito pequenos (ex.: 0 a 15) num aparelho praticamente sem RAM e com disco sobrando.

O que ensina: "Big-O de espaço" precisa dizer qual recurso está sendo medido. Economizar RAM empurrando o custo para o disco só esconde o problema.

10 · Matriz de adjacência n×n de auditoria

Com n = 1000, a matriz ocupa 125 KB, ≈ 488× o limite da Embarca. Com $n = 10^7$ seriam 12,5 TB de memória e ≈ 4,9 h de cálculo, informação que o problema nunca pediu.

A estratégia compara todos os pares (i, j) e guarda o resultado em n^2 bits, sempre, inclusive quando todos os valores são iguais. Então melhor, médio e pior caso são $\Theta(n^2)$ em tempo e em memória.

Item	$n = 10^3$	$n = 10^5$	$n = 10^7$
Comparações (n^2)	10^6	10^{10}	10^{14}
Tempo (× 0,18 ns)	≈ 0,18 ms	≈ 1,8 s	≈ 4,9 h
Memória (n^2 bits + 48 B de pilha)	≈ 125 KB	≈ 1,25 GB	≈ 12,5 TB
Comparado aos 256 B da Embarca	≈ 488×	≈ 4,9 milhões ×	≈ $4{,}9 \times 10^{10}$ ×

- **Embarca: inaceitável.** 48 B + $n^2/8$ bytes só cabe em 256 B com **n ⩽ 40** (40^2 = 1.600 bits). O sensor testa 10^3, 10^5 e 10^7.
- **Datacenter: inaceitável com n = 10^7.** ≈ 4,9 h (≈ 8.900× o limite de 2 s) e 12,5 TB de memória. Em Java, um `boolean[][]` usaria 1 byte por célula, 8× mais.
- **Desperdício:** metade da matriz é espelhada (M[i][j] = M[j][i]) e a diagonal é sempre verdadeira. Para decidir quem é repetido, basta saber se existe j < i com o mesmo valor.

Quando deixaria de ser absurda: na Datacenter, com n ⩽ ≈ 106 mil leituras o tempo cabe nos 2 s, e a memória fica em ≈ 1,4 GB. Ou seja, com n = 10^5 ela passaria raspando (1,8 s, 1,25 GB): só é reprovada porque o enunciado exige n = 10^7. E faria sentido se houvesse uma exigência real de auditar cada comparação (regulação, rastreabilidade).

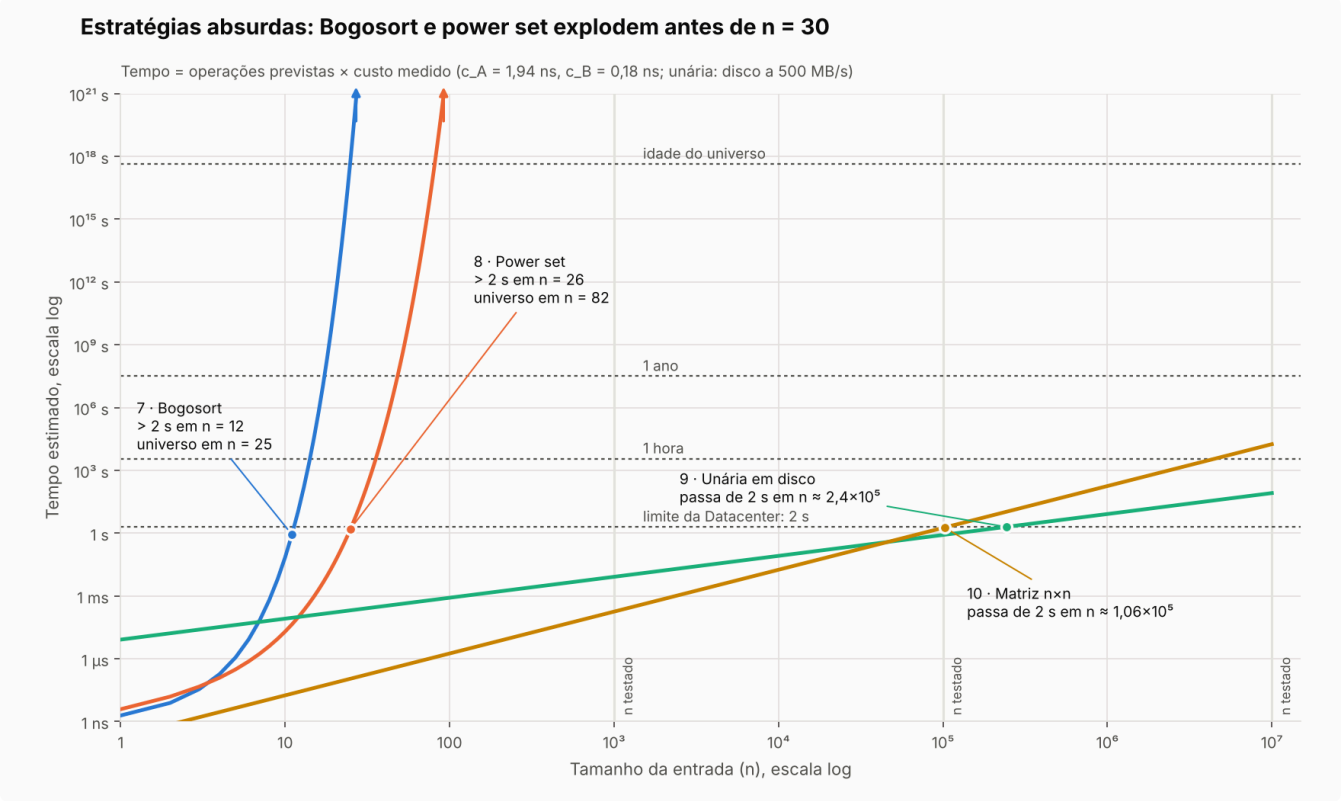
O que ensina: gastar memória "por segurança" sem que isso tenha sido pedido é tão errado quanto economizar memória demais quando ela sobra.

Quando deixariam de ser absurdas

As quatro passam em algum cenário, mas sempre com n muito menor que o testado: Bogosort e power set explodem antes de n = 30, e unária e matriz aguentam até ~10^5 leituras na Datacenter.

Estratégia	Maior n na Embarca (⩽ 256 B)	Maior n na Datacenter (< 2 s)	Iguala a idade do universo em
7 · Bogosort	Qualquer n na memória (48 B); para terminar em menos de 1 h, n ⩽ 14	n ⩽ 11 (≈ 0,85 s)	n = 25
8 · Power set	n ⩽ 3	n ⩽ 25 (≈ 1,6 s)	n = 82
9 · Unária em disco	Qualquer n na RAM (48 B), mas precisa de disco: com 1 MB de flash, n ⩽ ≈ 244	n ⩽ ≈ 244 mil (só a gravação)	—
10 · Matriz n×n	n ⩽ 40	n ⩽ ≈ 106 mil (≈ 1,4 GB)	—
Testado no trabalho	**10^3, 10^5 e 10^7**	**10^7**	

O gráfico abaixo mostra o tempo das quatro em função de n, com o limite de 2 s da Datacenter e a idade do universo como referências. Bogosort e power set sobem quase na vertical; unária e matriz crescem devagar e só cruzam o limite perto de n = 10^5.



Premissas e método

Todos os tempos são estimativas: número de operações previsto pela análise teórica × custo por operação medido nas duas estratégias implementadas.

Premissa	Valor	De onde vem
c_A (acessos espalhados, recursão, trocas)	1,94 ns por operação	Quicksort medido, n = 10^7: 844 ms ÷ 4,36 × 10^8 operações
c_B (varredura sequencial)	0,18 ns por operação	Força bruta medida, n = 10^7: 58,0 s ÷ 3,27 × 10^{11} operações
Régua de cada absurda	Bogosort e power set: c_A; matriz: c_B; unária: vazão do disco	Padrão de acesso mais parecido
Vazão do disco	500 MB/s (gravação sequencial)	Valor típico de SSD
Pilha	48 B por frame	Mesma estimativa dos programas Java
Valor médio de uma leitura	32.767,5	Média de 0..65535 (entrada uniforme)

Premissa	Valor	De onde vem
Idade do universo	$1{,}38 \times 10^{10}$ anos ≈ $4{,}4 \times 10^{17}$ s	Referência de escala

Os valores de c_A e c_B vêm de uma execução fora da máquina de referência. Ao rodar os programas na máquina de referência, recalculem c_A e c_B (o script extrapolacao.py faz isso) e atualizem os tempos. As quantidades que não dependem da máquina (tentativas, subconjuntos, bits, bytes, limiares de memória) não mudam.